CHESSKYS PRO · PESHMERGA EDITION



A chess platform that thinks, teaches and can't be cheated.

Strong in-browser engine. Server-checked online play. A personal coach that explains *why* a move was wrong. Built in Kurdistan, in four languages.

Custom 0x88 engine

11 bots · 400–2650

3 animated 3D coaches

Live online play

EN · كوردی · Kurmancî · العربية

THE PRODUCT IN ONE PAGE

What is Chesskys PRO?

A full chess platform that runs in the browser: play bots, play real people online, solve puzzles, get every game reviewed by a coach, climb a leaderboard and make friends — all wrapped in Kurdish culture and a theme system that makes it your own.

16.56M
perft nodes — move generator matches the published counts

depth 12
from the start position in a 3-second search

11
bot personalities, Elo 400 to 2650

12
time controls, 1 min to 45|15 and unlimited

♟ Play

- Vs 11 engine bots
- Worldwide humans-only matchmaking
- Private rooms with 6-character codes
- Friend challenges, spectators, rematch

🎓 Learn

- Post-game review with the coach
- Mistakes named by pattern (fork, hanging piece, missed mate…)
- Training exercises + verified puzzles + daily puzzle
- Hints, analysis mode, live eval bar

👥 Belong

- Real friendships, friends-only chat
- Quick chat and voice messages
- Leaderboard: global / friends
- Honor ranks from *Mountain Recruit* to *Supreme Peshmerga Grandmaster*

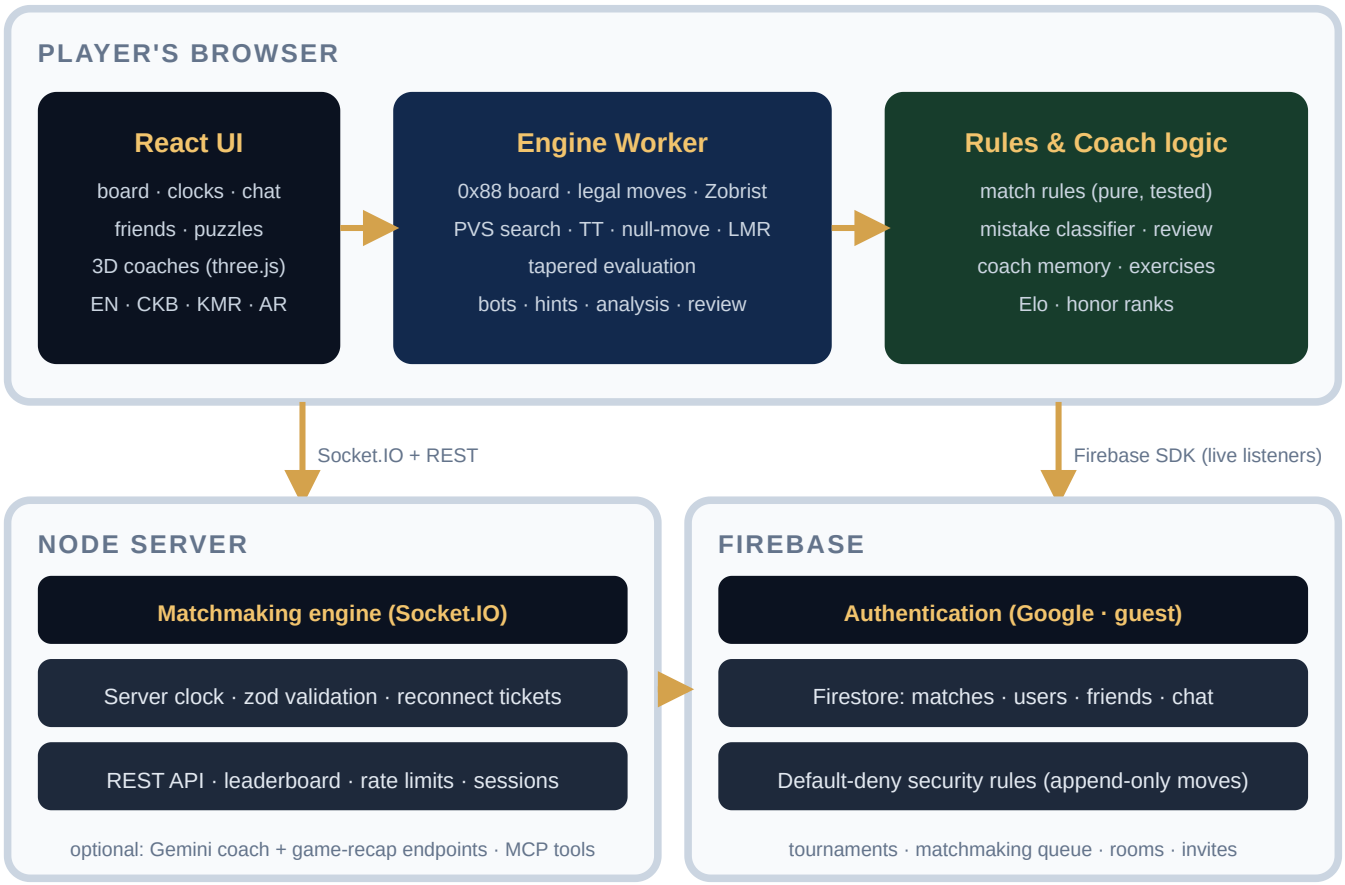
The stack, in plain words

Front end	React 19 · TypeScript · Vite 6 · Tailwind CSS 4 · three.js (3D coaches) · i18next (4 languages)
Engine	Hand-written chess engine in a Web Worker (board, search, evaluation, bot personalities); chess.js for SAN/PGN and rules display
Server	Node + Express + Socket.IO (`server.ts`, `server/`) — REST API, live matchmaking engine, rate limiting, optional Gemini endpoints
Data & auth	Firebase Authentication (Google, guests) + Cloud Firestore with a default-deny rules file
Ship	Docker files, Vercel config, GitHub Pages workflow, Firestore-rules deploy workflow, CI (typecheck + build)

HOW IT WORKS

Architecture

The heavy thinking happens on the player's own device; the shared truth lives on the server and in Firestore, behind rules. That split is why it feels instant *and* stays fair.



Three design decisions

- **Think locally.** The search runs in a Web Worker, so a 2650-rated bot never freezes the board, menus or clocks.
- **Keep rules pure.** Everything that decides "is this move legal / whose clock runs / is it over" is plain functions with unit tests — no UI, no database.
- **Never trust a peer.** Moves are checked by the server or re-derived from the move list by the other player's client.

Two live-play paths

- **Socket.IO engine** — server-owned game state, 6-character room codes, rated queue, precise clocks, reconnect tickets.
- **Firestore match sessions** — worldwide matching, friend challenges and spectating through live listeners, guarded by security rules.
- Both end in the same result flow: game-over screen, Elo change, review with the coach.

THE BRAIN

A real engine, not a toy

chess.js is excellent for rules and notation, but it manages only a few thousand positions a second — enough for a shallow search. Chesskys PRO ships its own allocation-free engine core for strength.

What's inside

- **0x88 board** with legal move generation and Zobrist hashing
- **Iterative-deepening PVS** with aspiration windows
- **Transposition table** — 1M entries (~16 MB)
- **Pruning & ordering:** null-move, late-move reductions, futility, killer + history moves, MVV-LVA, TT move first
- **Quiescence search** with delta pruning
- **Tapered evaluation:** material, piece-square tables, mobility, king safety, pawn structure, blended by game phase
- **Mate-aware** scores (shows #3, not just a number)

Proof, measured on this repo

Perft	All reference positions match — **16,564,718 nodes** in 4.1 s (~4.0M nodes/s). Move generation is provably correct on castling, en passant and promotion positions.
Tactics	**11 of 12** positions solved at 600 ms each (back-rank mate, Morphy mate in 2, smothered mate, Win-At-Chess set).
Depth	Start position, 3 s: **depth 12**, ~678k nodes, best move e2e4.
Types	Full TypeScript typecheck passes with zero errors.

Bots with personalities — and human mistakes

Early versions searched every strong bot to the same depth, so a "2300" played like a "1400". Now every bot has its own search budget plus an error model: evaluation *noise* (imprecision), a *blunder chance* with a capped loss (oversight), opening variety, and contempt (willingness to avoid draws). Weak bots really do hang pieces; strong ones punish them.

Elo	Bot	Style
400	🐣 Pawn Cadet	Sees one move ahead, hangs pieces often
700 · 1050	Zagros Scout · Knight Errant	Learning-curve opponents
1450 · 1500	Bishop Tactician · 🤡 Joker	Tactical / chaotic sacrifices
1800 · 1900	🦇 Batman · Rook Mastermind	Fortress defence / solid mastery
2000 · 2200	🍖 Luffy · ⚔️ Levi	Berserker attacker / surgical precision
2350 · 2650	Grandmaster DeepAI · Peshmerga Titan	Top of the ladder

FAIR PLAY

Online play that holds up under pressure

The classic failure of home-grown multiplayer is trusting the client. Chesskys PRO treats every move, clock and position as something to verify.

1 · Find
Humans-only worldwide queue, or a private room with a 6-character code.

▶

2 · Validate
Move schema checked, legality checked, turn checked, participants checked.

▶

3 · Clock
Time is computed from timestamps, not trusted from a client.

▶

4 · Record
Game ends, Elo updates, both sides can open the review.

Socket.IO matchmaking engine

- Room codes use a **31-character unambiguous alphabet** (no 0/O, 1/I/L)
- Queue pairs by rating: search window starts at ±50 and **widens by 100 every 5 s up to ±300**; ping gap ≤ 150 ms
- **High-resolution server clock** (`process.hrtime`) with flag-fall detection
- Signed **reconnect tickets** (HMAC-SHA256, 90 s) restore a game after a dropped connection
- Moves limited to 5 per second per socket; payloads schema-validated with zod
- Draw offers, rematch, resign, abort, abandonment handling
- Elo: K = 40 / 30 / 20 by experience, change capped at ±50, floor 100

Firestore match sessions

- **Server-corrected clocks** so a wrong system clock can't invent time
- **Pure rules module** (`matchRules.ts`) rejects illegal moves, moves out of turn, spectators moving, forged positions, flag falls
- **Anti-tamper:** the opponent's client replays the SAN move list and rejects any position that doesn't add up (red "Match state rejected" banner)
- **Rules:** default-deny; moves are append-only; finished games are locked
- A reload rebuilds the position, clocks and history from the stored session; draw by threefold repetition can be claimed
- Spectators by invite link; rematch with colours swapped; underpromotion supported

Fair-play monitoring

Timing analysis. The anti-cheat module flags move-time patterns no human produces: variance under 50 ms with average under 300 ms across 10+ moves. Five flags trigger a 7-day suspension in the module's logic. The browser also records frequent tab-switching as a soft signal.

Sensible limits at the door. Auth requests 5/min, chat 1/sec, friend requests 10/min, move validation 180/min per IP; chat text is sanitised server-side; sessions use signed, httpOnly cookies.

THE DIFFERENCE-MAKER

A coach that explains, not just scores

Most apps tell you a move was "?". Chesskys PRO tells you *what kind* of mistake it was — using facts on the board, never guesses.

Paul Morphy

Attacking, open-game spirit. Historical inspiration; biography uses documented facts only and the lines are written for the app, not quoted.

Capablanca

Clean, simple, endgame-minded. Same documented-facts rule.

Mamosta Zagros

An original Chesskys character — a Kurdish teacher in a headwrap and sash. All three are animated in 3D (three.js).

How a review works

Replay
The finished game is replayed move by move.

▶

Search
The engine searches every position where *you* moved (cached, cancellable, with progress).

▶

Measure
Winning-chance drop: ≥5 inaccuracy, ≥10 mistake, ≥15 blunder.

▶

Diagnose
Pattern named, best line and punishing line shown in SAN.

Seven mistake patterns recognised

- Allowed checkmate · Missed checkmate
- Hanging piece
- Allowed a fork · Missed a fork
- Missed free material
- Better move available (positional fallback)

Every label is decided by board facts — a capture that wins material, one piece attacking two targets, a forced mate — so the coach never names a tactic that isn't there. Back-rank mates are called out specifically.

Remembers you

- Coach memory records every exercise attempt and analysed move
- Derives your strength tier (beginner → master), weak themes and next lesson
- Tracks accuracy against the engine and improvement over your last 100 attempts
- Retry flow: try again → hint → reveal → similar exercises

Practice material

- **8** coach training exercises (mates, forks..., each line replay-tested)
- **25** verified puzzles in 10 categories; positions from real games; validated by script and, per the repo's notes, checked against Stockfish 16
- Daily puzzle with streak

In-game help

- Live eval bar (flips with the board, shows mate distance)
- Hints computed in the engine worker
- Move labels graded by winning chances: best / good / inaccuracy / mistake / blunder / brilliant
- Optional Gemini-powered manga-style game recap

CULTURE & COMMUNITY

Made in Kurdistan. Made for everyone.

Chesskys PRO celebrates Kurdish culture, the Peshmerga and the Zagros mountains — and still plays like a serious chess app anywhere in the world.

Four languages, right-to-left included

English · Sorani Kurdish (کوردی) · Kurmanji (Kurmancî) · Arabic (العربية), with dedicated RTL styling and a language selector.

Honor ranks reward fair play

Seven ranks driven by respect points: Mountain Recruit → Desert Scout → Zagros Vanguard → Peshmerga Guardian → Eagle of Kurdistan → Lion of Kurdistan → Supreme Peshmerga Grandmaster.

Social, with guard-rails

- Google sign-in for new accounts; guest mode to try instantly
- Real friendships (request → accept), friends-only chat
- Quick-chat phrases, voice clips (about 30 s), match challenges inside the chat
- Blocking and notifications; presence (online status)
- Global and friends leaderboards with week/month activity filters — only real, public, rated players are ranked

Make it yours

- UI theme presets (Brutalist, Bluesky, High-Contrast Dark, One Piece, Attack on Titan, Peshmerga, UKH, Midnight Cyber, Emerald Grandmaster) + custom themes and backgrounds
- Piece sets: Classic, Neo, Alpha, Vintage wood, Crystal Neon, FIDE 3D, One Piece, Attack on Titan, Batman, Peshmerga, UKH
- Board colour themes, sounds, legal-move hints, auto-queen, premoves option
- Voice move dictation (browser speech recognition) and keyboard shortcuts

Tournaments

Create a tournament with a name, player cap and time control; players register; a bracket view tracks rounds, and pairings spawn real match challenges.

Authoring & analysis

Players can author puzzles (the FEN and solution are validated before saving), like and solve each other's, and step through any game from the move list.

THE CASE

Why it's better

Compared with the usual way chess apps get built, each choice below solves a real, known weakness.

The usual approach	Chesskys PRO	What you gain
Rules engine only (a few thousand positions/s)	Dedicated engine, ~290k–4M nodes/s depending on the workload, perft-verified	Real strength, believable bots
Search on the UI thread	Web Worker search	No freezes while a bot thinks
All bots equally "perfect" at a depth	Per-bot budget + noise + blunder model	A ladder you can actually climb
Client sends a position, server trusts it	Server validation, SAN replay, append-only rules	Forged boards are rejected
Clocks from client timers	Server-corrected / hrtime clocks, increments only to the mover	Agreeing clocks, honest flags
Dropped connection = lost game	Reconnect tickets, state restore, abandonment handling	Fair handling of bad networks
Review = engine score "?"	Pattern-named mistakes, best line, coach memory	You learn *why*
Open chat for anyone	Friends-only chat, sanitised text, voice clips	Safer community
English-only, left-to-right	4 languages incl. 2 Kurdish dialects, RTL	Underserved players welcome

Quality gates behind the claims

Test suites, all passing
Engine (perft + tactics), 50 multiplayer-rule assertions, 30 platform checks, 15 hardening checks, coach & puzzle suites, game-rule and opening suites, plus Firestore-rules tests.

Typecheck & CI
`tsc --noEmit` is clean. CI typechecks and builds on every push and pull request.

Deployable several ways
Node server, Docker files, Vercel, GitHub Pages, and a workflow that publishes Firestore rules.

STRAIGHT TALK

Accuracy notes

This overview was written from the repository itself and from test runs on it. Where something isn't what a quick skim of the old README would suggest, it is stated here.

Topic	What's true in the code today
Which stack runs?	`npm run dev` / `npm start` run `server.ts` (Express + Socket.IO + Vite) with Firebase Auth/Firestore. The README, `docker-compose.yml`, `backend/`, `schema.sql` and `API.md` describe an earlier PostgreSQL + Redis + JWT design; those files remain in the repository but aren't what the current app is wired to.
Redis	The server's leaderboard cache uses `ioredis-mock` (in-memory), and the matchmaking state lives in the server process. A multi-instance deployment would need a shared store.
Bot Elo values	These are design targets from each bot's search budget and error model, not ratings measured against rated human pools.
Engine speed	Perft ran at ~4.0M nodes/s on this machine; real searches ran at roughly 150k–480k nodes/s. Results vary by device.
Anti-cheat	Timing/variance flags and the suspension logic are implemented in a shared module; engine-correlation features named in some docs are partly aspirational. Treat it as a first line, not a complete fair-play system.
Firestore rules	The rules file is strict by default but keeps a documented exception so unauthenticated guests can play; they are not live until deployed to the Firebase project.
Worldwide opponents	The Worldwide modal pairs **humans only**. The code also contains a pool of named "challenger" personas that are engine opponents, used only by the non-strict matchmaking modes.
AI features	Gemini coaching and game recap work only when `GEMINI_API_KEY` is configured on the server; the deterministic engine-based coach and review need no key.
Demo edition	`DEMO_FEATURES_UI_UX.md` defines a browser-only showcase for UKH Induction Day with simulated opponents and mock data; this PDF describes the full platform.

How the numbers were obtained. Dependencies were installed with `npm ci`, then `npm run lint`, `test:engine`, `test:multiplayer`, `test:multiplayer-platform`, `test:phase1`, `test:coach` and `test:rules` were run on the current code. Every one passed; the one tactical miss (WAC.002) is reported above as 11 of 12.

Chesskys PRO

Kurdish chess, online — engine, coach, community.